

Отчёт по лабораторной работе №8

Оптимизация

Кузнецова София Вадимовна

Информация

- Кузнецова София Вадимовна
- Оптимизация

Основная цель работы – освоить пакеты Julia для решения задач оптимизации.

Julia – высокоуровневый свободный язык программирования с динамической типизацией, созданный для математических вычислений. Эффективен также и для написания программ общего назначения. Синтаксис языка схож с синтаксисом других математических языков, однако имеет некоторые существенные отличия. Для выполнения заданий была использована официальная документация Julia.

Выполнение лабораторной работы

Примеры из лабораторной работы

```
j: # Определение объекта модели с именем model:
model = Model(GLPK.Optimizer)

j: A JuMP Model
  solver: GLPK
  objective_sense: FEASIBILITY_SENSE
  num_variables: 0
  num_constraints: 0
  Names registered in the model: none

j: # Определение переменных x, y и граничных условий для них:
@variable(model, x >= 0)
@variable(model, y >= 0)

j: y

j: # Определение ограничений модели:
@constraint(model, 6x + 8y >= 180)
@constraint(model, 7x + 12y >= 120)

j:

$$7x + 12y \geq 120$$


j: # Определение целевой функции:
@objective(model, Min, 17x + 20y)

j: 12x + 20y

j: # Вызов функции оптимизации:
optimize!(model)

j: # Определение причины завершения работы оптимизатора:
termination_status(model)

j: OPTIMAL::TerminationStatusCode = 1

j: # Демонстрация первичных результирующих значений переменных x и y:
@show value(x);
@show value(y);

value(x) = 15.000000000000002
value(y) = 1.2499999999999999

j: # Демонстрация результата оптимизации:
@show objective_value(model);

objective_value(model) = 285.0
```

Рис. 1: Линейное программирование

Примеры из лабораторной работы

```
vector_model = Model(GLPK_Optimizer)

# 3-й этап: Model
{
  solver: GLPK
  objective_sense: FEASIBILITY_SENSE
  num_variables: 4
  num_constraints: 4
  Names registered in the model: none
}

# Определение исходных данных:
A = [ 1 1 9 5;
      3 5 0 0;
      2 0 13;
      0 1 3 5]
b = [1; 3; 5]
c = [1; 3; 5; 2]

# 4-элемент Vector(Int64):
1
3
5
2

# Определение порядка переменных:
@variable(vector_model, x[1:4] >= 0)

# 4-элемент Vector(VariableRef):
x[1]
x[2]
x[3]
x[4]

# Определение ограничений модели:
@constraint(vector_model, A * x .== b)

# 3-элемент Vector{ConstraintRef{Model, MathOptInterface.ConstraintIndex{MathOptInterface.ScalarAffineFunctionInterface, EqualTo{Float64}}, ScalarShape}}:
x[1] = x[2] + 9 * x[3] + 5 * x[4] = 1
3 * x[1] + 5 * x[2] + 0 * x[3] + 0 * x[4] = 3
2 * x[1] + 0 * x[2] + 13 * x[3] + 0 * x[4] = 5

# Определение целевой функции:
@objective(vector_model, Min, c * x)

# Вывод функции оптимизации:
optimize!(vector_model)

# Определение причины завершения работы оптимизатора:
termination_status(vector_model)

OPTIMAL::TerminationStatusCode = 1

# Определение результата оптимизации:
@show objective_value(vector_model);
objective_value(vector_model) = 4.9230769230769225
```

Рис. 2: Векторизованные ограничения и целевая функция оптимизации

Примеры из лабораторной работы

```
8]: # Контейнер для хранения данных об ограничениях на количество
# потребленных калорий, белков, жиров и соли:
category_data = JuMP.Containers.DenseAxisArray{
    [1800 2200;
     91 Inf;
     0 65;
     0 1779],
    ["calories", "protein", "fat", "sodium"],
    ["min", "max"]})

8]: 2-dimensional DenseAxisArray{Float64,2,...} with index sets:
  Dimension 1, ["calories", "protein", "fat", "sodium"]
  Dimension 2, ["min", "max"]
And data, a 4x2 Matrix{Float64}:
1800.0 2200.0
 91.0   Inf
   0.0   65.0
   0.0 1779.0

9]: # массив данных с наименованиями продуктов:
foods = ["hamburger", "chicken", "hot dog", "fries", "macaroni",
        "pizza", "salad", "milk", "ice cream"]

9]: 9-element Vector{String}:
 "hamburger"
 "chicken"
 "hot dog"
 "fries"
 "macaroni"
 "pizza"
 "salad"
 "milk"
 "ice cream"

8]: # Массив стоимости продуктов:
cost = JuMP.Containers.DenseAxisArray{
    [2.49, 2.89, 1.59, 1.89, 2.09, 1.99, 2.49, 0.89, 1.59],
    foods)

8]: 1-dimensional DenseAxisArray{Float64,1,...} with index sets:
  Dimension 1, ["hamburger", "chicken", "hot dog", "fries", "macaroni", "pizza", "salad", "milk", "ice cream"]
And data, a 9-element Vector{Float64}:
 2.49
 2.89
 1.59
 1.89
 2.09
 1.99
 2.49
 0.89
 1.59
```

Рис. 3: Оптимизация рациона питания

Примеры из лабораторной работы

```
21: # Массив данных о содержании калорий, белков, жиров и соли в продуктах питания:
food_data = JuMP.Containers.DenseAxisArray{
  [410 24 26 730;
  420 32 18 1100;
  560 20 32 1800;
  300 4 19 270;
  320 12 10 930;
  320 13 12 820;
  320 31 12 1230;
  100 0 2.5 125;
  330 0 10 180],
  foods,
  ["calories", "protein", "fat", "sodium"]}

22: 2-dimensional DenseAxisArray{Float64,2,...} with index sets:
  Dimension 1, ["hamburger", "chicken", "hot dog", "fries", "macaroni", "pizza", "salad", "milk", "ice cream"]
  Dimension 2, ["calories", "protein", "fat", "sodium"]
And data, a 9x4 Matrix{Float64}:
 410.0 24.0 26.0 730.0
 420.0 32.0 18.0 1100.0
 560.0 20.0 32.0 1800.0
 300.0 4.0 19.0 270.0
 320.0 12.0 10.0 930.0
 320.0 13.0 12.0 820.0
 320.0 31.0 12.0 1230.0
 100.0 0.0 2.5 125.0
 330.0 0.0 10.0 180.0

23: # Определение задачи оптимизации с именем model:
model = Model{GLPK.Optimizer}

24: A JuMP Model
  + solver: GLPK
  + objective_sense: FEASIBILITY_SENSE
  + num_variables: 0
  + num_constraints: 0
  Names registered in the model: none

25: # Определение массива
categories = ["calories", "protein", "fat", "sodium"]

26: 4-element Vector{String}:
 "calories"
 "protein"
 "fat"
 "sodium"

27: # Определение переменных:
@variables(model, begin
  category_data{c, "max"} <= nutrition[c <- categories] <=
  category_data[c, "max"]
  # Сколько покупок продуктов:
  buy{foods} >= 0
end)

28: (1-dimensional DenseAxisArray{VariableRef,1,...} with index sets:
  Dimension 1, ["calories", "protein", "fat", "sodium"]
And data, a 4-element Vector{VariableRef})
```

Рис. 4: Оптимизация рациона питания

Примеры из лабораторной работы

```
25]: # Определение целевой функции:
@objective(model, Min, sum(cost[f] * buy[f] for f in foods))

25]: 2.49buy[hamburger] + 2.89buy[chicken] + 1.5buy[hotdog] + 1.89buy[fries] + 2.09buy[macaroni] + 1.99buy[pizza] + 2.49buy[salad] + 0.89buy[milk] + 1.59buy[ice cream]

27]: # Определение ограничений модели:
@constraint(model, [c in categories],
sum(food_data[f, c] * buy[f] for f in foods) == nutrition[c])

27]: 1-dimensional DenseColArray{ConstraintRef{Model, MathOptInterface.ConstraintIndex{MathOptInterface.ScalarAffineFunction{Float64, MathOptInterface.EqualTo{Float64}}}, ScalarShape{1},...}} with index sets:
      Dimension 1, ["calories", "protein", "fat", "sodium"]
And data, a 4-element Vector{ConstraintRef{Model, MathOptInterface.ConstraintIndex{MathOptInterface.ScalarAffineFunction{Float64, MathOptInterface.EqualTo{Float64}}}, ScalarShape{1}}}:
 -nutrition[calories] + 410 buy[hamburger] + 420 buy[chicken] + 560 buy[hot dog] + 380 buy[fries] + 320 buy[macaroni] + 20 buy[pizza] + 320 buy[salad] + 100 buy[milk] + 330 buy[ice cream] = 0
 -nutrition[protein] + 24 buy[hamburger] + 32 buy[chicken] + 20 buy[hot dog] + 4 buy[fries] + 12 buy[macaroni] + 15 buy[pizza] + 31 buy[salad] + 0 buy[milk] + 0 buy[ice cream] = 0
 -nutrition[fat] + 26 buy[hamburger] + 10 buy[chicken] + 32 buy[hot dog] + 19 buy[fries] + 10 buy[macaroni] + 12 buy[pizza] + 12 buy[salad] + 2.5 buy[milk] + 10 buy[ice cream] = 0
 -nutrition[sodium] + 730 buy[hamburger] + 1100 buy[chicken] + 1000 buy[hot dog] + 270 buy[fries] + 930 buy[macaroni] + 20 buy[pizza] + 1230 buy[salad] + 125 buy[milk] + 180 buy[ice cream] = 0

29]: # Вызов функции оптимизации:
JuMP.optimize!(model)
term_status = JuMP.termination_status(model)

29]: OPTIMAL::TerminationStatusCode = 1

30]: hcat(buy_data, JuMP.value.(buy_data))

30]: 9x2 Matrix{AffExpr}:
 buy[hamburger]  0.6045138888888899
 buy[chicken]   0
 buy[hot dog]   0
 buy[fries]     0
 buy[macaroni]  0
 buy[pizza]     0
 buy[salad]     0
 buy[milk]      6.978138888888885
 buy[ice cream] 2.5913194444444445
```

Рис. 5: Оптимизация рациона питания

Примеры из лабораторной работы

```
71: using DelimitedFiles
using CSV

82: # Чтение данных:
passportdata = readin(joinpath("passport-index-dataset", "passport-index-matrix.csv"), ',')

83: 200x200 Matrix{Any}:
"Passport"      "Albania"      -1  "Afghanistan"
"Albania"      "Afghanistan"  -1  "e-visa"
"Albania"      "Albania"      -1  "visa required"
"Algeria"      "e-visa"      90  "visa required"
"Andorra"      "e-visa"      90  "visa required"
"Angola"      "e-visa"      90  "visa required"
"Antigua and Barbuda" 90  "visa required"
"Argentina"    90  "visa required"
"Armenia"      90  "visa required"
"Australia"    90  "visa required"
"Austria"      90  "visa required"
"Azerbaijan"   90  "visa required"
"Bahamas"      90  "visa required"
"United Arab Emirates" 90  "visa required"
"United Kingdom" 90  "visa required"
"United States" 90  "visa required"

84: # Задача оптимизации:
cntr = passportdata[2:end,:];
vf = (x -> typeof(x) == Int64 || x == "VF" || x == "VQA" ? 1 : 0) * passportdata[2:end,2:end];

72: # Оптимизация с помощью решателя:
model = Model{GLPK.Optimizer}()

73: A JuMP Model
├ solver: GLPK
├ objective_sense: FEASIBILITY_SENSE
├ num_variables: 0
├ num_constraints: 0
└ Names registered in the model: none

85: # Переопределение и установка функции:
@variable{model, pass[i,1:length(cntr)], Bin}
@constraint{model, [j=1:length(cntr)], sum vf[i,j] - pass[i] for i in 1:length(cntr)} == 1
@objective{model, Min, sum(pass)}

86: pass1 + pass2 + pass3 + pass4 + pass5 + pass6 + pass7 + pass8 + pass9 + pass10 + pass11 + pass12 + pass13 + pass14 + pass15 + p

87: # Вывод функции оптимизации:
JuMP.optimize!(model)
termination_status(model)

88: OPTIMAL::TerminationStatusCode = 1

89: # Вывод результатов:
print(JuMP.objective_value(model), " passports:", join(cntr[findall(JuMP.value.(pass) == 1)], ", "));
34.0 passports:Afghanistan, Australia, Bahrain, Cameroon, Comoros, Congo, Djibouti, Dominican Republic, Eritrea, Guinea
issai, Hong Kong, Iraq, Kenya, Kuwait, Liberia, Libya, Madagascar, Maldives, Mauritania, Morocco, Nepal, New Zealand, N
th Korea, Palestine, Papua New Guinea, Qatar, Saudi Arabia, Singapore, Somalia, South Sudan, Spain, Syria, Turkmenistan
```

Рис. 6: Путешествие по миру

Примеры из лабораторной работы

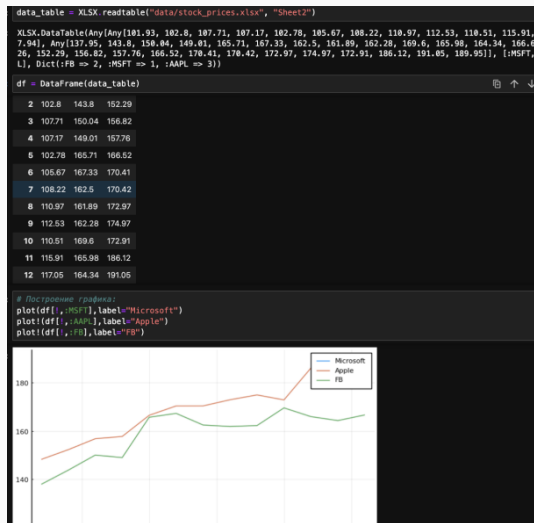


Рис. 7: Портфельные инвестиции

Примеры из лабораторной работы

```
prices_matrix = Matrix(df)
# Вычисляем матрицу доходности за период времени:
M1 = prices_matrix[1:end-1,1]
M2 = prices_matrix[2:end,1]
# Матрица доходности:
R = (M2 - M1) ./ M1
# Матрица рисков:
risk_matrix = cov(R)
# Проверка положительной определенности матрицы рисков:
isposdef(risk_matrix)
# Список из индексов по компаниям:
r = mean(R,dims=1)[1:]
# Список переменных:
x = Variable(length(r))
# Оптимизируем:
problem = minimize(Convex.quadform(x,risk_matrix),[sum(x) == 1; r'*x == 0.02; x >= 0])
# Решаем задачу:
solve!(problem, SCS.Optimizer)

Expression graph
minimize
└─ L = (convex; positive)
   └─ [1;1]
      └─ quad_form (convex; positive)
         └─ norm2 (convex; positive)
            └─ L = -
               └─ [1.0;]

subject to
└─ on constraint (affine)
   └─ L = (affine; real)
      └─ sum (affine; real)
         └─ L = -
            └─ [-2;1]
               └─ a constraint (affine)
                  └─ L = (affine; real)
                     └─ a (affine; real)

x

Variable
size: (3, 1)
sign: real
realtype: affine
id: 524_598
value: [0.077487709795831287, 0.11086771003983582, 0.806527626687796]

sum(x).value
1.800882434677944

r'*x.value
1e1 adjoint(::Vector{Float64}) with eltype Float64:
0.019947233188531883

x.value == 1.000

3x1 Matrix{Float64}:
 77.48770979583128
116.86771003983582
 806.527626687796
```

Рис. 8: Портфельные инвестиции

Примеры из лабораторной работы

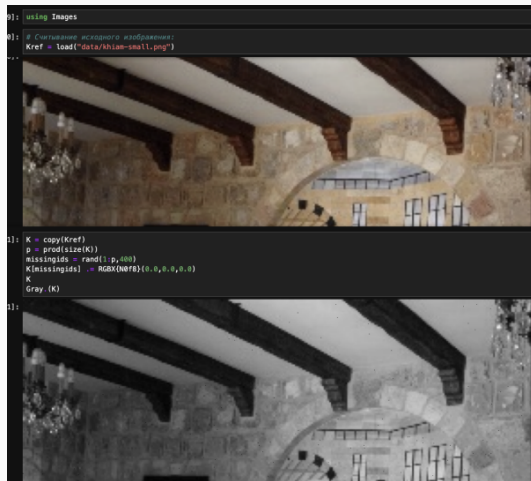


Рис. 9: Восстановление изображения

Примеры из лабораторной работы

```
1 # Maximize entropy:
2 Y = Float64{Gray(K)};

3 correctids = findall(Y[:,1,:].is0)
4 X = Convex.Variable{size(Y)}
5 problem = minimize(nuclearnorm(X))
6 problem.constraints += X[correctids] == Y[correctids]

7 Warning: Concatenating collections of constraints together with '+' or '+= to produce a new list
8 recreated. Instead, use 'vcat' to concatenate collections of constraints.

9 1-element Vector{Constraint}:
10 == constraint (affine)
11  └─ (affine; real)
12     └─ index (affine; real)
13        └─ 952x968 real variable (id: 117-378)
14           └─ 913528x1 Matrix{Float64}

15 using SCS

16 # Maximize problem:
17 solve!(problem, SCS.Optimizer)

18 Info: [Convex.jl] Compilation finished: 24.4 seconds, 62.768 GiB of memory allocated

19 =====
20 SCS v3.2.7 - Splitting Conic Solver
21 (c) Brendan O'Donoghue, Stanford University, 2012
22 =====
23 problem: variables n: 1828829, constraints m: 2742349
24 cones: z: primal zero / dual free vars: 913528
25        l: linear vars: 3
26        s: psd vars: 1828828, ssize: 1
27 settings: eps_abs: 1.0e-04, eps_rel: 1.0e-04, eps_infeas: 1.0e-07
28          alpha: 1.50, scales: 1.00e-01, adaptive_scale: 1
29          max_iters: 10000, normalizer: 1, rho_x: 1.00e-06
30          acceleration_lookback: 10, acceleration_interval: 10
31          compiled with openmp parallelization enabled
32 lin-sys: spars-direct-mss-qldl
33          nnz(A): 2744261, nnz(P): 0
34 =====
35
36 1) (show norm(float.(Gray.(Kref)))-X.value)
37 norm(float.(Gray.(Kref)) - X.value) = 0.05095558310232877
38 0.05095558310232877
39
40 1) (show norm(-X.value))
41 norm(-X.value) = 417.58335756316654
42 417.58335756316654
```

Рис. 10: Восстановление изображения

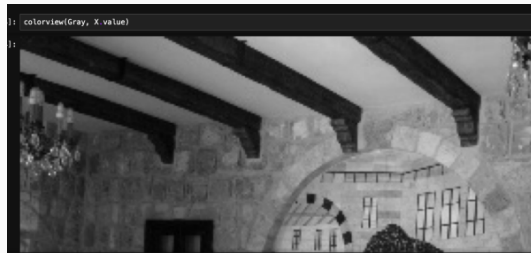


Рис. 11: Восстановление изображения

Задания для самостоятельной работы

```
1: # Определение объекта модели с именем model:
model = Model(GLPK.Optimizer)

2: # A JuMP Model
   + solver: GLPK
   + objective_sense: FEASIBILITY_SENSE
   + num_variables: 0
   + num_constraints: 0
   + Names registered in the model: none

3: # Определение переменных x, y и граничных условий для них:
@variable(model, x1 <= 10)
@variable(model, x2 >= 0)
@variable(model, x3 >= 0)

4: x3

5: # Определение ограничений модели:
@constraint(model, -x1 + x2 + 3x3 <= -5)
@constraint(model, x1 + 3x2 - 7x3 <= 10)

6:
   x1 + 3x2 - 7x3 ≤ 10

7: # Определение целевой функции:
@objective(model, Max, x1 + 2x2 + 5x3)

8: x1 + 2x2 + 5x3

9: # Выход функции оптимизации:
optimize!(model)
# Определение причины завершения работы оптимизатора:
termination_status(model)

10: OPTIMAL::TerminationStatusCode = 1

11: # Демонстрация первичных результирующих значений переменных x и y:
@show value(x1);
@show value(x2);
@show value(x3);

value(x1) = 10.0
value(x2) = 2.1875
value(x3) = 0.9375
```

Рис. 12: 1 Задание - Линейное программирование

Задания для самостоятельной работы

```
# Определение объекта модели с именем model:
vector_model = Model(GLPK.Optimizer)

A JuMP.Model()
+ solver: GLPK
+ objective_sense: FEASIBILITY_SENSE
+ num_variables: 0
+ num_constraints: 0
+ names registered in the model: none

A = [-1 1 2; 1 1 -1]
b = [-5; 10]
c = [1; 2; 3]

3-element Vector{Int64}:
 1
 2
 3

@variable(vector_model, x[1:3] >= 0)

3-element Vector{VariableRef}:
 x[1]
 x[2]
 x[3]

# Определение ограничений модели:
@constraint(vector_model, A * x .<= b)
@constraint(vector_model, x[1] <= 10)

# Определение целевой функции:
@objective(vector_model, Min, c' * x)

# Вывод функции оптимизации:
optimize!(vector_model)
# Определение текущего состояния работы оптимизатора:
termination_status(vector_model)

OPTIMAL::TerminationStatusCode = 1

# Демонстрация полученных результатов значений переменных x и y:
@show value(x1);
@show value(x2);
@show value(x3);

value(x1) = 10.0
value(x2) = 2.1875
value(x3) = 0.9375

# Демонстрация результата оптимизации:
@show objective_value(vector_model);
objective_value(vector_model) = 5.0
```

Рис. 13: 2 Задание - Линейное программирование. Использование массивов

Задания для самостоятельной работы

```
using Random

m = 10
n = 5

Random.seed!(42)
A = rand(m, n)
b = rand(m)

x = Variable(n)

objective = sumsquares(A*x - b)
constraints = [x >= 0]

problem = minimize(objective, constraints)

Problem statistics
  problem is DCP          : true
  number of variables    : 1 (5 scalar elements)
  number of constraints   : 1 (5 scalar elements)
  number of coefficients : 67
  number of atoms        : 6

Solution summary
  termination status : OPTIMIZE_NOT_CALLED
  primal status      : NO_SOLUTION
  dual status        : NO_SOLUTION

Expression graph
  minimize
  └─ qol (convex: positive)
     └─ + (affine; real)
        └─ * (affine; real)
           └─ -
```

```
solve!(problem, SCS.Optimizer)
```

```
[ Info: [Convex.jl] Compilation finished: 1.36 seconds, 286.170 MiB of memory allocated
```

```
-----
SCS v3.2.7 - Splitting Conic Solver
(c) Brendan O'Donoghue, Stanford University, 2012
-----
problem: variables n: 6, constraints m: 17
cones:   l: linear vars: 5
         q: soc vars: 12, qsize: 1
settings: eps_abs: 1.0e-04, eps_rel: 1.0e-04, eps_infeas: 1.0e-07
          alpha: 1.50, scale: 1.0e-01, adaptive_scale: 1
          max_iters: 10000, normalizer: 1, rho_x: 1.0e-06
          acceleration_lookback: 10, acceleration_interval: 10
          compiled with openmp parallelization enabled
lin-sys: sparse-direct-and-qdldl
          nnz(A): 57, nnz(P): 0
-----
iter | pri res | dua res | gap   | obj   | scale | time (s)
-----
[ ]: println("Статус задачи: ", problem.status)
Статус задачи: OPTIMAL
```

Рис. 14: 3 Задание - Выпуклое программирование

Задания для самостоятельной работы

```
num_sections = 5
num_applications = 1000

min_capacity = [100, 100, 220, 100, 100]
max_capacity = [250, 250, 220, 250, 250]

Random.seed!(42)
priorities = [shuffle(1:250, 10000, 10000)] for _ in 1:num_applications
priorities = hcat(priorities...)
priority_weights = sum(priorities, dims=1)

1x5 Matrix{Int64}:
3851232 4091162 4181199 4031198 3851209

model = Model{GLPK.Optimizer}()

@variables(model, begin
    100 <= x1 <= 250
    100 <= x2 <= 250
    x3 <= 220
    100 <= x4 <= 250
    100 <= x5 <= 250
end)

(x1, x2, x3, x4, x5)

@constraint(model, x1+x2+x3+x4+x5==1000)
@objective(model, Min, x1 + priority_weights[1] *
    x2 + priority_weights[2] *
    x3 + priority_weights[3] *
    x4 + priority_weights[4] *
    x5 + priority_weights[5])

3851232x1 + 4091162x2 + 4181199x3 + 4031198x4 + 3851209x5

optimize!(model)
termination_status(model)

OPTIMAL::TerminationStatusCode = 1

println("Результаты")
println("x1: ", value(x1))
println("x2: ", value(x2))
println("x3: ", value(x3))
println("x4: ", value(x4))
println("x5: ", value(x5))
println("Оптимальное значение целевой функции: ", objective_value(model))

Результаты:
x1: 100.0
x2: 100.0
x3: 220.0
x4: 100.0
x5: 260.0
Оптимальное значение целевой функции: 3.9344805e9
```

Рис. 15: 4 Задание - Оптимальная рассадка по залам

Задания для самостоятельной работы

```
: coffee = ["Raf", "Cappuccino"]
products = ["grains", "milk", "sugar"]

cost = JuMP.Containers.DenseAxisArray{
    [400, 300],
    coffee
}
coffee_data = JuMP.Containers.DenseAxisArray{
    [40 140 5;
     30 120 0],
    coffee,
    products
}

store = JuMP.Containers.DenseAxisArray{[500, 2000, 40],products}

: 1-dimensional DenseAxisArray{Int64,1,...} with index sets:
   Dimension 1, ["grains", "milk", "sugar"]
   And data, a 3-element Vector{Int64}:
      500
     2000
       40

: model = Model{GLPK.Optimizer}
@variables(model, begin
    buy[coffee] >= 0
end)
@objective(model, Max, sum(cost[c] * buy[c] for c in coffee))
@constraint(model, [p in products],
    sum(coffee_data[c, p] * buy[c] for c in coffee) <= store[p])
@constraint(model, coffee_data["Raf", "sugar"] * buy["Raf"] == 40)

:
                                     5buyRaf = 40

: JuMP.optimize!(model)
term_status = JuMP.termination_status(model)
hcat(buy.data, JuMP.value.(buy.data))

: 2×2 Matrix{AffExpr}:
 buy[Raf]      8
 buy[Cappuccino] 6
```

Рис. 16: 5 Задание - План приготовления кофе

Вывод

В результате выполнения данной лабораторной работы я освоила пакеты Julia для решения задач оптимизации.

Спасибо за внимание!